AES Benchmarking on Raspberry Pi 5

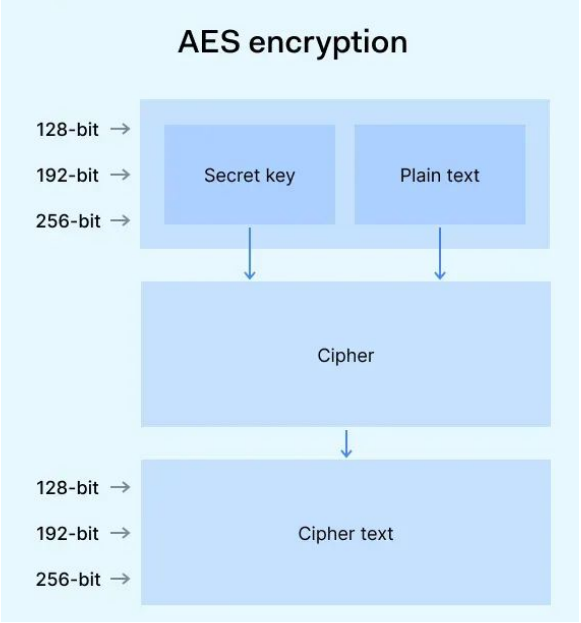
Group H: Zach Agle, Abigail Jeffie, Juliana Neemeh, Jacob Dominguez

Cal Poly Pomona

Project Details

<u>Goal:</u> Analyze how architectural implementation choices affect encryption performance on the Raspberry Pi 5.

We will benchmark hardware-accelerated AES on the Raspberry Pi 5 using two independent measurement methods: perf paired with OpenSSL and cryptsetup.



<u>Measured Values:</u>

- Throughput (MB/s)
- CPU cycles
- Instructions executed
- Instructions per Cycle
- Cache miss rate (%)
- Additional metic will collected

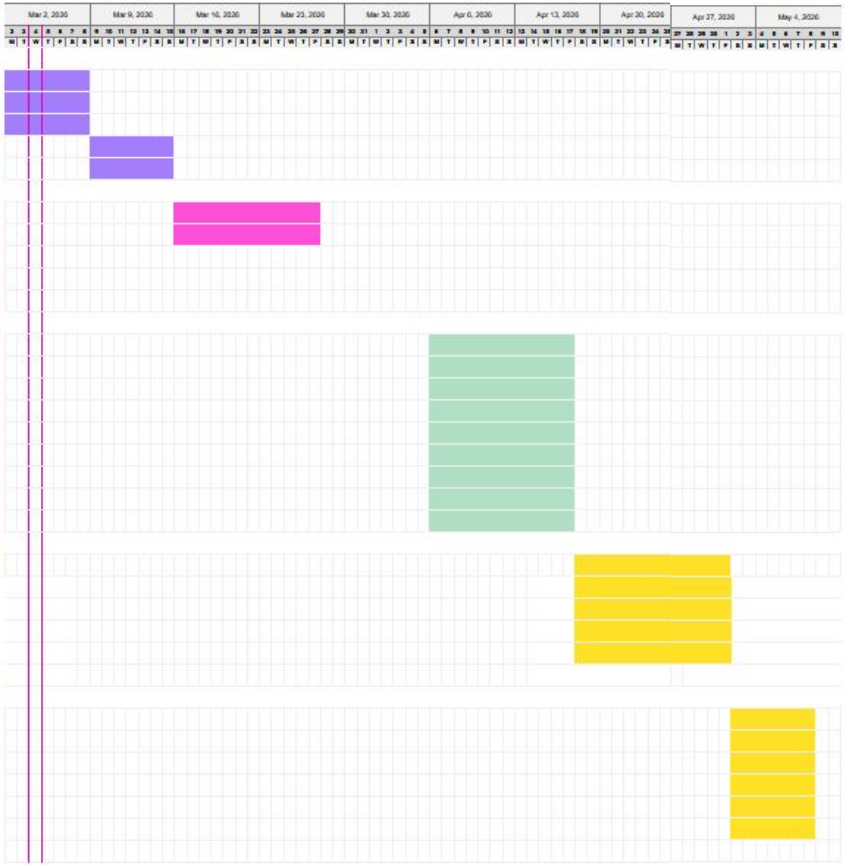
4300 Project

Project start: Wed, 3/4/2026

Display week: 1

TASK	ASSIGNED TO	PROGRESS	START	END
Background Research and Planning				
Study Raspberry Pi Architecture			3/2/26	3/8/26
Review AES Algorithms			3/2/26	3/8/26
Review OpenSSL Implementation Details			3/2/26	3/8/26
Define Benchmarking Methods			3/9/26	3/15/26
Finalize Experimental Design			3/9/26	3/15/26
Planning and Design				
Implement hardware aes			3/16/26	3/27/26
Implement software aes			3/16/26	3/27/26
Experiment Setup				
Design Benchmark Program Architecture			4/6/26	4/17/26
Implement AES Encryption Module			4/6/26	4/17/26
Implement Timing & Measurement Logic			4/6/26	4/17/26
Integrate perf Metrics Collection			4/6/26	4/17/26
Verification & Output Validation			4/6/26	4/17/26
Define Test Matrix (Key Sizes & Input Sizes)			4/6/26	4/17/26
Configure CPU Settings (Disable Scaling)			4/6/26	4/17/26
Create Automated Test Scripts			4/6/26	4/17/26
Pilot Test Runs			4/6/26	4/17/26
Data Collection and Analysis				
Run Hardware and Software AES Benchmarks			4/18/26	5/1/26
Collect perf Metrics			4/18/26	5/1/26
CPU Cycles & IPC Analysis			4/18/26	5/1/26
Cache Miss Analysis			4/18/26	5/1/26
Organize Data,Calculate Statistics, Finish Graphics			4/18/26	5/1/26
Materials Collection and Presentation				
Write Introduction, Background and Methodology of IEEE Report			5/2/26	5/8/26
Write Results & Analysis			5/2/26	5/8/26
Write Conclusion & Future Work			5/2/26	5/8/26
Create Slide Deck			5/2/26	5/8/26
Prepare Demonstration Video			5/2/26	5/8/26
Final Review & Submission			5/2/26	5/8/26

Insert new row ABOVE this one

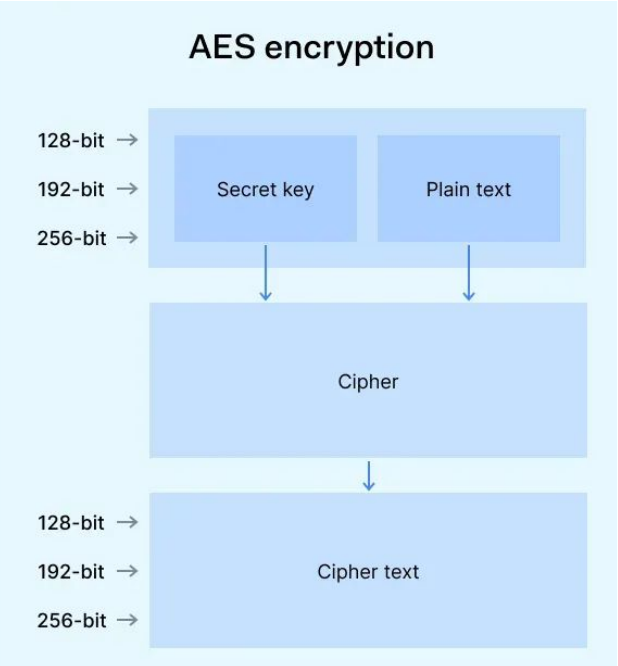


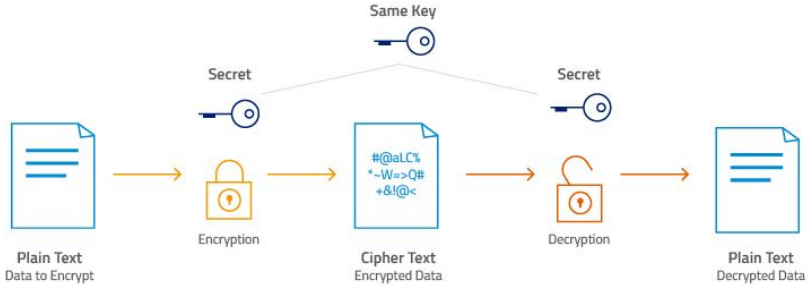
Tasks:

- Research
- Setup
- Implement hardware
- Implement software
- Benchmark
- Analyze data
- Write report
- Demo video
- Slides

Advanced Encryption Standard

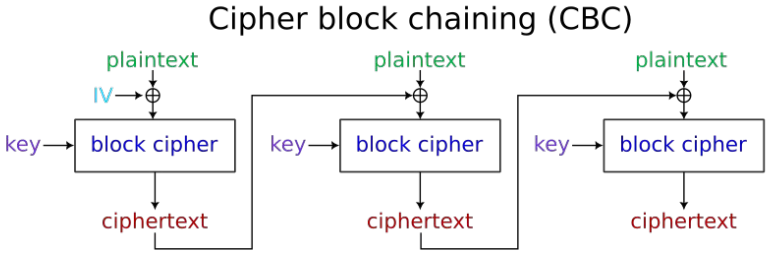
- A symmetric-key encryption algorithm used to secure digital information.
- Encrypts data by converting readable plaintext into unreadable ciphertext, and decrypts it back using the same secret key.
- Transforms plaintext into ciphertext through repeated rounds of substitution, shifting, mixing, and key-based transformations
- Uses keys lengths of 128, 192, or 256 bits (longer keys → stronger security)
- Used in applications like online banking, messaging apps, and protecting sensitive files
- Example:
  - Plaintext: HELLO → Ciphertext: XJQZP → Decrypted bacl to HELLO

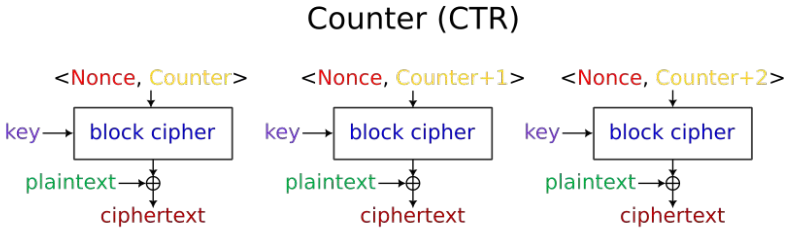




AES Modes Being Benchmarked:

- **CBC (Cipher Block Chaining):** encrypts each data block using the previous block's output.
- **CTR (Counter Mode):** encrypts each data block independently using a counter and a unique value.
- **GCM (Galois/Counter Mode):** extends CTR with authentication, verifying data integrity alongside encryption in a single operation.





Benchmark Tools

- Benchmarks AES encryption performance using different libraries, modes, and key sizes.
  - **Cryptsetup**
    - Modes: CBC, CTR
    - Key sizes: 128, 192, and 256-bit
  - **OpenSSL**
    - Modes: CBC, CTR, GCM
    - Key sizes: 128, 192, and 256-bit
- Collect CPU performance data using stat perf:
  - Data collected: cycles, instructions, cache references and misses, branch references and misses
- Converts CPU metrics into per-second rates.
- Monitors system temperature and checks permissions.

LUKS
Linux Unified Key Setup

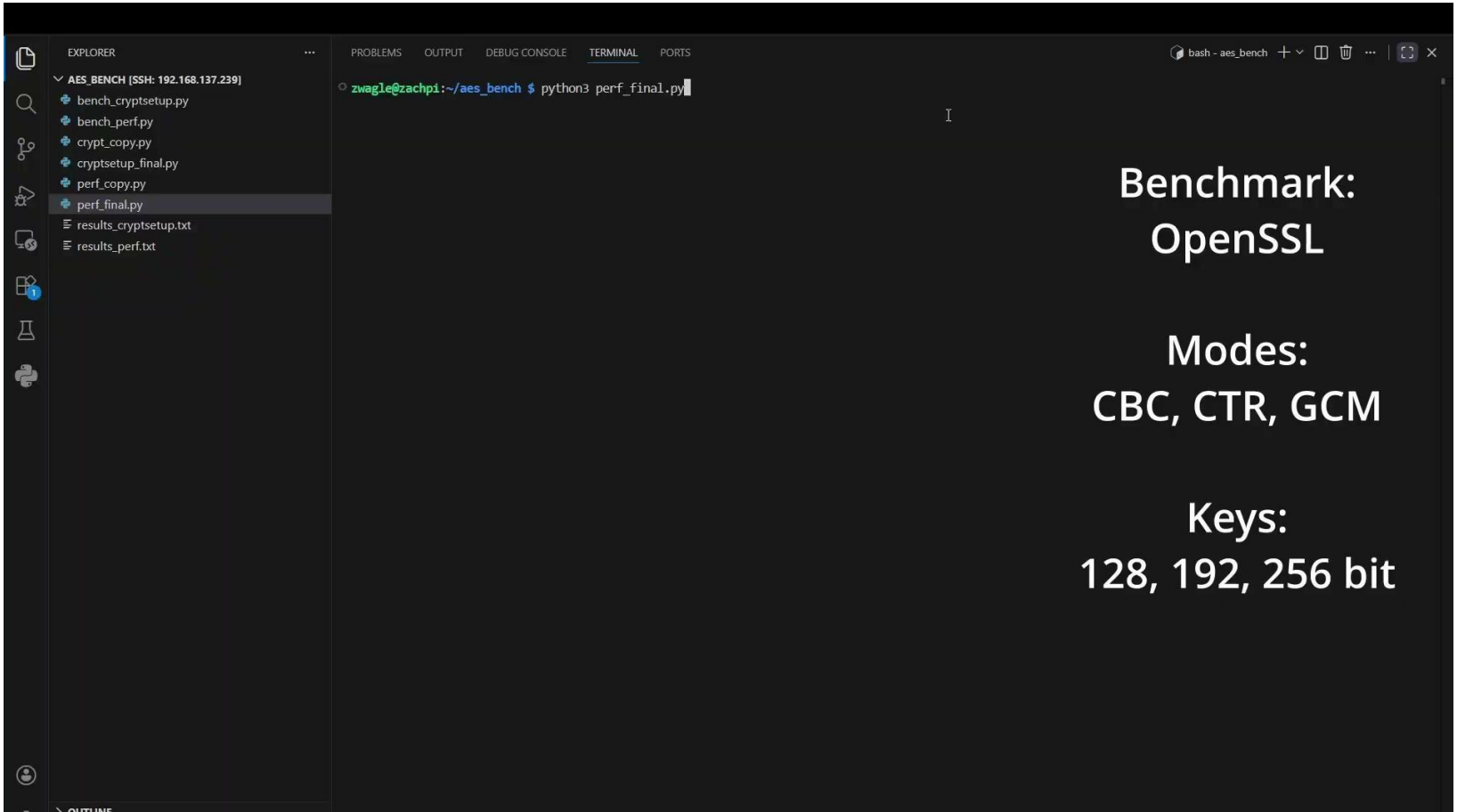
Source Code Explained

- Both Python scripts, **cryptsetup.py** and **OpenSSL.py**, run AES benchmarks wrapped in the Linux perf stat.
- Both scripts divide perf counter totals by perf's reported elapsed time, converting raw values to per-second rates for display.
- For each AES variant, cryptsetup.py runs **perf stat cryptsetup benchmark** and streams the throughput live from stdout.
- For each AES variant, OpenSSL.py runs **perf stat openssl speed -evp** and displays a post-processed output after the operation completes.

OpenSSL run() Operation

```
def run(label, cipher):
    print(f"\n  {'─'*54}\n  {label:<20}  temp: {temp()}\n  {'─'*54}")   # Print variant header and PI temperature
    result = subprocess.run(                                              # Run perf stat wrapping openssl speed
        ["perf", "stat", "-e", EVENTS, "openssl", "speed", "-elapsed", "-seconds", DURATION, "-evp", cipher],
        stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)      # Capture stdout, redirect stderr into stdout stream
    output = clean(result.stdout)                                         # Strip OpenSSL build header
    norm, elapsed = normalize(output)                                     # Normalize counters to per-second rates
    for l in output.splitlines():                                         # Print throughput lines only
        if any(x in l for x in ["MiB/s", "MB/s", "bytes"]):
            print(f"  {l.strip()}")
    print(f"\n  perf counters (per second, elapsed: {elapsed:.2f}s):")   # Print counter header with elapsed time
    for l in norm:                                                        # Print per-second counter lines
        if "/sec" in l: print(l)
```

Source Code Demo



Project Blockers

perf permission restrictions:

- Linux by default restricts perf to userspace only.
- Cryptsetup AES executes almost entirely in kernel mode and the values read near zero.
- Setting **perf_event_paranoid=-1** removes the restriction, allowing perf to count events across both userspace and kernel mode.

cryptsetup does not support GCM:

- cryptsetup benchmark targets disk encryption, while GCM is used in communication protocols, making it outside cryptsetup scope.
- OpenSSL supports GCM through its own userspace library, directly accessing hardware AES acceleration without relying on a kernel crypto API.
- Thus, GCM encryption could not be compared with both benchmark tools.

Terminal Command Fix:

sudo sh -c 'echo -1 > /proc/sys/kernel/perf_event_paranoid'

Comparing Data

cryptsetup benchmark

CBC Mode (Encrypt / Decrypt)
128-bit: 1141 / 2264 MiB/s
192-bit: 1054 / 2042 MiB/s
256-bit: 947 / 1832 MiB/s

CTR Mode (Encrypt / Decrypt)
128-bit: 2170 / 2179 MiB/s
192-bit: 1896 / 1897 MiB/s
256-bit: 1717 / 1712 MiB/s

CPI (Cycles/Instruction)
CBC: 0.65–0.66 | CTR: 0.46–0.48

Cache Miss Rate
CBC: ~2.6% | CTR: ~2.9%

perf stat + OpenSSL

CBC Mode (CPI)
128: 0.50 | 192: 0.48 | 256: 0.48

CTR Mode (CPI)
128: 0.33 | 192: 0.33 | 256: 0.34

GCM Mode (CPI) — auth overhead
128: 0.36 | 192: 0.35 | 256: 0.36

Cache Miss Rate (all modes)
≈0.00% — near-perfect L1/L2 hit rate

Branch Miss Rate
CBC/CTR: ≈0.01% | GCM: 0.00–0.28%

Key Takeaways

- CTR is the fastest mode (>2 GiB/s at 128-bit); CBC decrypt is ~2x encrypt due to hardware parallelism
- OpenSSL + perf shows near-zero cache misses (<0.01%) vs cryptsetup's 2.6–2.9% — longer OpenSSL test windows warm caches fully
- GCM CPI (0.35–0.36) falls between CBC and CTR — GHASH authentication adds instructions but stays highly efficient

Thank You

Any Questions?